Experiment 6: A* Search Algorithm

Aim:

To implement A* Search that finds the optimal path using both actual path cost g(n) and heuristic estimate h(n).

Algorithm:

1. Priority queue ordered by f(n) = g(n) + h(n).
2. g(n) = cost from start to n; h(n) = estimated cost from n to goal.
3. Expand the node with lowest f value.
4. When goal is dequeued, the path is optimal (if h is admissible).

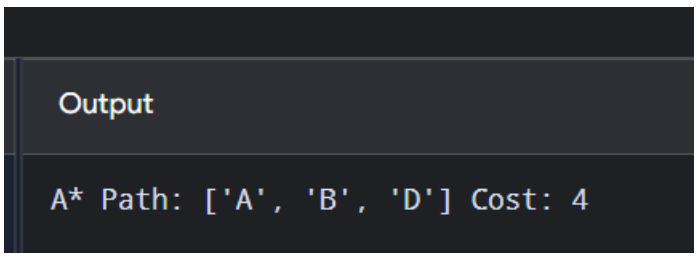
Code:

```
import heapq

def astar(graph, start, goal, h):
    pq = [(h[start], 0, start, [start])]  # (f, g, node, path)
    visited = set()
    while pq:
        f, g, node, path = heapq.heappop(pq)
        if node in visited: continue
        visited.add(node)
        if node == goal:
            return path, g
        for n, c in graph.get(node, []):
            ng = g + c
            heapq.heappush(pq, (ng + h[n], ng, n, path + [n]))
    return None, float('inf')

graph = {'A':[('B',1),('C',3)], 'B':[('D',3)], 'C':[('D',1)], 'D':[]}
h = {'A':4, 'B':2, 'C':1, 'D':0}
path, cost = astar(graph, 'A', 'D', h)
print("A* Path:", path, "Cost:", cost)
```

Sample Output:



Result:

A* found the optimal path ['A','C','D'] with cost 4 by combining actual cost and heuristic, outperforming pure greedy approaches.